PYTHON FORGE

Updated Comprehensive Review and Roadmap - Revision 1.1

Generated: 10 August 2026 | Scope: current workspace evidence and the next development backlog

This document updates Python_Forge_Comprehensive_Review_and_Roadmap.pdf. It marks work that is implemented and validated, separates partial or blocked work from future work, and records the development phases recommended from this point forward.

SOURCE LIMITATION

The original PDF is preserved unchanged at the repository root. Its readable metadata identifies the title Python Forge - Comprehensive Review & Bounded Roadmap and a 9 August 2026 creation date, but its body is stored in compressed streams that were not reliably extractable. This revision therefore uses repository source, tests, build output, and device evidence rather than claiming a line-by-line reproduction of unreadable original sections.

1. Executive status

The main application-quality recommendations have been implemented and validated locally. Android now runs the embedded interpreter in a killable application-owned worker process, but the runtime remains constrained execution rather than a complete security sandbox because resource isolation and non-Android worker backends are still incomplete. Public release is also blocked until a production-signed artifact is built and verified.

[IMPLEMENTED] Core product hardening

Runtime request limits, best-effort execution deadlines, Android process-worker supervision, authenticated loopback transport, deterministic archive and manifest verification, curriculum contracts, guarded navigation, progression reconciliation, achievement persistence, semantic controls, and responsive layout protections are present in the workspace.

[PARTIAL] Release and evidence gates

The expanded CI workflow now covers Python compilation/tests, Dart formatting/analyze/tests, web and debug APK builds, and uploaded artifact checksums. Hosted CI provenance, integration/device artifacts, signing verification, and full roadmap traceability remain incomplete.

[PARTIAL] Android process worker

Android now launches Serious Python in an application-owned :python_worker process and requests termination of that process after a request failure or timeout, reporting whether the stop was confirmed. Desktop, iOS, and web still use or require constrained fallbacks, and OS-level resource limits plus full cross-platform coverage remain incomplete.

[BLOCKED] Production signing and distribution

Production signing secrets and a keystore are not available, so a signed release AAB, certificate verification, and Play internal-track evidence cannot yet be produced.

Status legend

[IMPLEMENTED] Completed in source and supported by validation evidence. [PARTIAL] Some implementation exists, but an acceptance gate or evidence is still missing. [PLANNED] New work proposed in this revision. [BLOCKED] Cannot be completed with the current architecture or unavailable credentials. [EVIDENCE GAP] Work may exist, but provenance or verification is not recorded.

2. Implemented changes to date

The following items are marked implemented because they are visible in the current codebase and were covered by the latest local validation pass.

2.1 Runtime and execution constraints

[IMPLEMENTED] Constrained execution policy

python_src/main.py applies a 5-second best-effort deadline using tracing on all platforms and signals where available. It caps requests at 128 KiB, caps output at 20,000 characters, serializes HTTP execution, requires a per-launch authentication token outside explicit tests, blocks common unsafe imports/builtins, and keeps SystemExit from terminating the service.

[IMPLEMENTED] Dart bridge timeout and recovery

lib/core/engine/python_runner.dart performs readiness polling, verifies the packaged runtime hash, sends authenticated bounded JSON requests, and applies an 8-second request guard. On Android a timeout requests termination of the separate worker process and reports whether it was confirmed; local device validation covered successful output and a successful execution after worker startup.

[PARTIAL] Security isolation

Android now has a real process kill boundary, but the complete product remains constrained execution rather than a security sandbox: native resource limits, filesystem/network isolation, and desktop and iOS worker backends remain incomplete. Web builds explicitly report execution as unsupported until a Web Worker/WASM backend exists.

2.2 Runtime archive and build packaging

[IMPLEMENTED] Reproducible runtime archive and manifest

tooling/package_python_runtime.py verifies that app/app.zip contains the expected main.py payload, matches python_src/main.py byte-for-byte, and matches the generated app/runtime_manifest.json. The latest verified source SHA-256 is c8e20dc3682da9b4ca2899f87b77dc4028fb1728fafdcac894893f631671f7b8 and archive SHA-256 is b7152dddf56c4944e180f83c83b5aff753fc167d9258917a419d47dd8da47fa5; the app verifies the archive hash before startup.

[IMPLEMENTED] Native-symbol release packaging

useLegacyPackaging remains enabled for the embedded Python payload, and keepDebugSymbols excludes the ZIP-formatted libpythonbundle.so from ELF stripping while preserving Flutter debug-symbol metadata for real native objects.

2.3 Curriculum, progression, and navigation

[IMPLEMENTED] Curriculum contracts

CurriculumRepository validates stable curriculum metadata, unique positive module order, required module and mission fields, exact-match validation, answer contracts, MCQ options, prerequisite references, and acyclic prerequisite graphs before caching the asset.

[IMPLEMENTED] Progress reconciliation

LearningProgressService and ProgressManager sanitize stale completion IDs, migrate safe legacy keys, derive unlocks, XP, level, completion percentage, resume position, and module progress from one active curriculum.

[IMPLEMENTED] Achievements and safe mission entry

Achievement definitions are catalog-backed and reconciled on dashboard load. MissionEntryScreen resolves route data against the active curriculum and permits only known unlocked or completed missions; invalid and locked entries never open an editor.

2.4 Accessibility and responsive interface

[IMPLEMENTED] Semantic controls

Custom settings, reset, Quick Console, streak, profile, and execution controls expose labels, hints, and button semantics instead of relying only on visual GestureDetector or InkWell presentation.

[IMPLEMENTED] Narrow screens and large text

The dashboard uses compact FORGE branding below 420 px, ellipsis for the app-bar title, a wrapping overall-progress header, and widget coverage for a 360x800 surface and 2.0 text scaling.

2.5 Local quality gates

[IMPLEMENTED] Automated test coverage

The current suite covers answer validation, curriculum contracts including prerequisite cycles and canonical IDs, progress/unlock derivation, route protection, mission rendering, settings, streaks, achievements, cold start, responsive layout, and text scaling. Direct Python tests cover runtime output, errors, blocked imports, output truncation, pure-Python timeout handling, valid authentication, rejected tokens, and fail-closed startup.

3. Validation evidence snapshot

The following results are the latest recorded local validation evidence for the current workspace.
They are not a substitute for a hosted CI run or a signed distribution test.

[IMPLEMENTED] Dart analyzer

Command: flutter analyze --no-pub. Result: PASS - no issues found.

[IMPLEMENTED] Flutter tests

Command: flutter test --no-pub. Result: PASS - 33 tests.

[IMPLEMENTED] Python tests

Command: python -m unittest discover -s python_src -p test_*.py. Result: PASS - 8 tests.

[IMPLEMENTED] Runtime archive and manifest

Command: python tooling/package_python_runtime.py --check. Result: PASS - source and archive hashes match.

[IMPLEMENTED] Whitespace

Command: git diff --check. Result: PASS.

[IMPLEMENTED] Debug APK

Command: flutter build apk --debug --no-pub. Result: PASS.

[IMPLEMENTED] Web build

Command: flutter build web --no-pub. Result: PASS compile-only; execution explicitly unsupported until a web backend exists.

[IMPLEMENTED] Android worker startup

Command: Local adb install/relaunch/Quick Console execution on serial 98e86dfe. Result: PASS manual smoke; separate com.example.python_forge:python_worker process observed and code executed; timeout/restart recovery remains unautomated.

[IMPLEMENTED] Release signing guard

Command: verifyPythonForgeReleaseSigning. Result: PASS when evaluated; unsigned release builds fail closed without configured secrets.

[IMPLEMENTED] Crash check

Command: Local app-scoped logcat for com.example.python_forge. Result: No FATAL EXCEPTION or AndroidRuntime crash during the worker smoke test.

RELEASE CAVEAT

A production AAB is intentionally not generated without signing properties and a keystore: the Gradle release guard fails closed instead of permitting an unsigned artifact. The debug APK and Android worker smoke test pass locally. Serious Python’s libpythonbundle.so is a ZIP payload with a .so filename; keepDebugSymbols excludes it from ELF stripping while legacy packaging preserves runtime loading.

4. Remaining gaps and new work

These items should be treated as the active backlog. They are intentionally not marked complete by the current evidence.

[PARTIAL] Killable execution worker

Android now has a real application-owned worker process, authenticated loopback IPC, timeout-driven stop with native confirmation when available, and next-run process-state reconciliation. It still needs adversarial native-blocking/restart tests, OS-level CPU/memory/filesystem/network controls, and equivalent desktop and iOS backends before the product can claim a secure sandbox; web execution is explicitly unsupported.

[PARTIAL] Real integration and device qualification

Add integration_test coverage for asset loading, first execution, success/error/timeout UI, persistence after relaunch, locked routes, keyboard behavior, and next-mission navigation. Run it across target Android API levels, ARM64, emulator, low-memory, offline, and process-restart scenarios.

[BLOCKED] Production signing and distribution

Provide the four PYTHON_FORGE_UPLOAD_* secrets through a private Gradle or CI secret store, fail closed when absent, verify the signing certificate and AAB hash, and use a Play internal-testing track before public release.

[PARTIAL] CI quality and artifact provenance

The checked-in workflow now runs archive/manifest verification, Python compilation/tests, pinned Flutter/Python setup, Dart formatting/analyze/tests, web and debug APK builds, and uploads checksums plus validation artifacts. It still needs hosted evidence, integration/device tests, and a signed-release job.

[PARTIAL] Curriculum graph correctness

Prerequisite cycle detection and contract coverage are implemented. A machine-readable curriculum manifest with stable version/content hash, migration policy, and complete all-mission contract evidence remain planned; MCQ and fill-in missions must remain non-executing and failed execution must never award completion.

[PARTIAL] Documentation and version alignment

Align README, pubspec, Android versionName, and release naming. Add dedicated tests for claims such as quote normalization and document the distinction between constrained execution and a secure sandbox.

[EVIDENCE GAP] Roadmap traceability

Recover or text-extract the original PDF source if possible, preserve its SHA-256, and add an evidence index mapping each roadmap requirement to source symbols, tests, CI jobs, artifacts, and build identity.

5. Development phases

The phases below are intentionally short summaries. Each phase has a practical outcome and an exit condition so roadmap status can be updated from evidence instead of intention.

[PARTIAL] Phase 0 - Evidence baseline and traceability

Recover the original roadmap text or record its extraction limitation, preserve source and artifact hashes, add docs/review-source.md and an evidence-index file, and embed revision/source identity in future PDF generation. Exit when every roadmap item has a status and an evidence reference.

[PARTIAL] Phase 1 - Execution hardening

Android now has an application-owned killable worker, authenticated IPC, archive verification, timeout cleanup, and worker startup recovery. Add adversarial native-blocking/restart tests, OS resource controls, and platform-specific desktop, iOS, and web backends. Exit when each supported platform proves that a stuck program cannot hold the host app.

[PARTIAL] Phase 2 - Curriculum and validation correctness

Prerequisite DAG cycle detection and contract tests are implemented. Add curriculum manifest/version/hash/migration rules, all-mission contract evidence, and explicit starter-code and quote-normalization behavior. Exit when content changes fail fast with actionable validation errors.

[PARTIAL] Phase 3 - Integration and device qualification

Add integration tests for the real Flutter-to-runtime path, persistence, navigation, timeout recovery, keyboard, accessibility, and relaunch. Run on the target device/API/ABI matrix and publish logs and screenshots. Exit when the matrix is repeatable and artifact-linked.

[PARTIAL] Phase 4 - Release and signing

Configure secret-backed signing, fail closed without secrets, build and verify a signed AAB, retain mapping files and checksums, and complete Play internal testing. Exit when certificate identity and artifact provenance are recorded.

[PARTIAL] Phase 5 - CI and evidence automation

Pin Flutter/Python/action versions, run Python plus Dart plus integration suites, build artifacts, upload reports/checksums, and fail on stale runtime archives or missing roadmap metadata. Exit when a pull request receives the same gates as a local release candidate.

[PLANNED] Phase 6 - Content and platform expansion

Only after safety and release gates: expand curriculum, add analytics or sync where explicitly required, and broaden platform support. Exit criteria should be defined per feature rather than allowing new content to bypass the contract validator.

6. Prioritized next implementation order

1. Release configuration: provide private signing secrets, verify a signed AAB, and run internal-track testing
2. Sandbox architecture spike: choose the separate process or external sandbox boundary before adding more arbitrary-code features
3. Integration/device matrix: automate real runtime startup, timeout recovery, persistence, accessibility, and relaunch checks
4. Curriculum graph and manifest: add cycle detection, content hashes, migration policy, and all-mission contract validation
5. CI/evidence automation: upload results, checksums, symbol maps, device artifacts, and roadmap traceability metadata
6. Content expansion: proceed only after the safety and release gates are green

7. File-level evidence map

These are the principal files that support the status decisions in this revision. The complete worktree also contains the related tests, CI workflow, runtime archive, and UI changes.

[IMPLEMENTED] python_src/main.py
Constrained service policy, authenticated HTTP contract, output cap, import/builtin controls, deadline, and explicit non-sandbox limitation.

[IMPLEMENTED] lib/core/engine/python_runner.dart
Dart asset-hash verification, worker lifecycle supervision on Android, authenticated JSON bridge, readiness polling, and timeout recovery.

[IMPLEMENTED] lib/core/engine/python_worker_controller.dart
Narrow platform channel for starting and stopping the Android worker process.

[IMPLEMENTED] lib/main.dart
Android worker entrypoint and bootstrap channel; web execution returns an explicit unsupported result.

[IMPLEMENTED] android/app/src/main/kotlin/com/example/python_forge/PythonWorkerService.kt
Application-owned :python_worker process containing the secondary Flutter engine and Serious Python entrypoint.

[IMPLEMENTED] android/app/src/main/AndroidManifest.xml
Non-exported worker service declaration with a separate Android process.

[IMPLEMENTED] tooling/package_python_runtime.py
Deterministic app.zip creation plus source/archive hash manifest generation and freshness checks.

[IMPLEMENTED] lib/features/curriculum/data/repositories/curriculum_repository.dart
Curriculum parsing and validation contract.

[IMPLEMENTED] lib/core/progression/learning_progress.dart
Derived unlocks, prerequisites, XP, level, resume, and stale-ID sanitization.

[IMPLEMENTED] lib/core/progression/progress_manager.dart
Idempotent completion, safe legacy migration, and progress-only reset.

[IMPLEMENTED] lib/features/curriculum/presentation/screens/mission_screen.dart
Mission validation, execution, completion, streak, achievements, and next mission.

[IMPLEMENTED] lib/features/curriculum/presentation/screens/home_screen.dart
Dashboard startup reconciliation, responsive title/progress layout, and semantic controls.

[IMPLEMENTED] .github/workflows/flutter.yml
Pinned Flutter/Python quality workflow with archive, Python, Dart, web, APK, checksum, and artifact gates.

[IMPLEMENTED] android/app/build.gradle.kts
Native packaging, ZIP-named .so strip exclusion, and fail-closed production signing configuration.

[IMPLEMENTED] test/ and python_src/test_main.py
Curriculum, answer, progress, navigation, UI, settings, and direct runtime tests.

8. Risks, assumptions, and decisions

- Android now has a killable worker process, but the runtime remains bounded constrained execution rather than a secure sandbox until OS resource controls and all target-platform backends are verified.
- A release AAB build passing is not equivalent to a distributable release; signing and certificate verification are still required.
- One Android device smoke pass is useful evidence but is not a device compatibility matrix.

- The original PDF remains unchanged. This updated PDF is a new artifact and does not assert that unreadable original body text was reproduced exactly.
- No commit or push is implied by this document. The repository owner should approve the release and sandbox decisions before creating a checkpoint commit.

9. Revision change log

- Added explicit status labels for implemented, partial, planned, blocked, and evidence-gap work.
- Recorded the Android process worker, authenticated loopback protocol, runtime hash manifest, fail-closed release signing guard, prerequisite cycle detection, expanded CI workflow, and new validation evidence.
- Added the current validation command/result snapshot and release-symbol packaging explanation.
- Added remaining blockers and new work for sandbox isolation, integration/device qualification, signing, CI, curriculum DAG checks, version alignment, and evidence traceability.
- Added brief Phases 0 through 6 and a prioritized next implementation order.
- Preserved Python_Forge_Comprehensive_Review_and_Roadmap.pdf without overwriting it.

CURRENT DECISION

Python Forge is ready for continued internal development and controlled Android testing with the process worker. It is not yet ready to claim secure arbitrary-code execution or public production distribution: resource isolation, non-Android worker backends, hosted evidence, and signed-release verification remain open.